

Instruction Set Architecture (ISA)

- ↳ Interface between the hardware and software
- ↳ ISA helps to utilize the hardware to its complete ability through software instructions.
- ↳ an instructor
- ↳ helps in defining the operations, mode of operation supported, addressing, storage related input.
- ↳ there are many processors and there could be more ISAs

* $A = B + C \Rightarrow (\text{ADD } A, B, C)$

↓ ↙ ↘
destination register source register

3 most common types of ISAs are (based on the no. of operands)

1. Stack - The operands are implicitly on top of the stack.
2. Accumulator - One operand is implicitly the accumulator.
3. General Purpose Register (GPR) - All operands are explicitly mentioned; they are registers or memory locations.

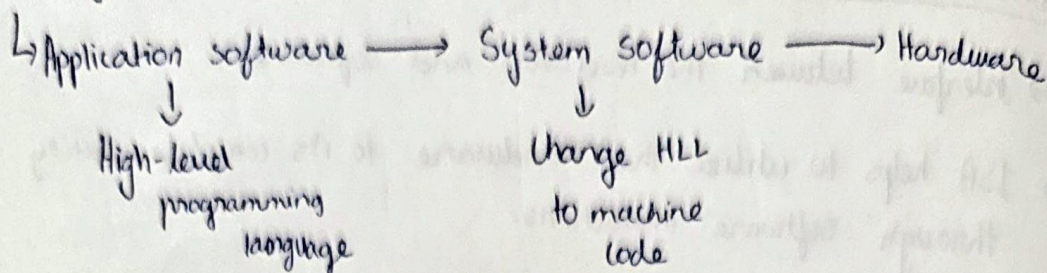
Understanding Performance

- ↳ Algorithm
- ↳ Programming language, compiler, architecture
- ↳ Processor and memory system
- ↳ I/O system (including OS)

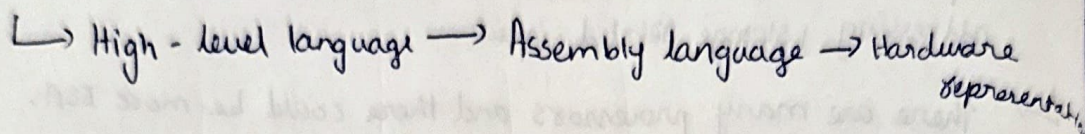
Architecture or Organization

- ↳ Instruction set
- ↳ Control signals
- ↳ Instruction size
- ↳ Addressing mode

Below your program

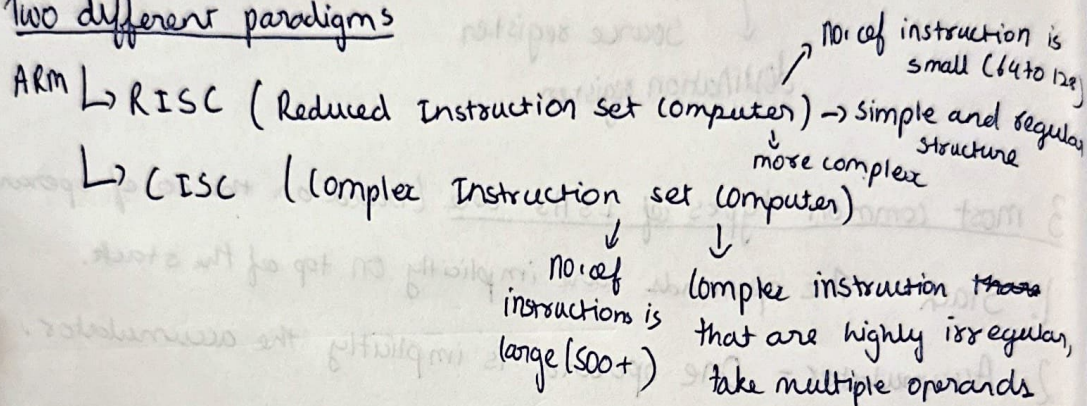


Level of Program code



Designing an ISA

* Two different paradigms



Stored program concept

↳ The instruction and data is stored together in memory

Uses of Registers

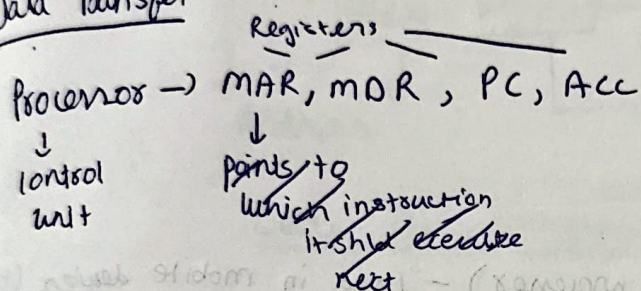
- ↳ A CPU contains set of registers
- ↳ Named as storage locations
- ↳ Arithmetic / logical instruction use register as input operands

Read/Write operations

- The write operation - transfers the address of the desired word to the address lines, transfers the data bits ~~from~~ to the memory
- Activates the write control line
- The read operations - transfers the address of desired word to the address lines
- Activates the read control line

memory enable	Read/Write	memory operation
0	X	None
1	0	Write to selected word
1	1	Read from selected word

Data Transfer



Data input line → provides the info to be store into the memory

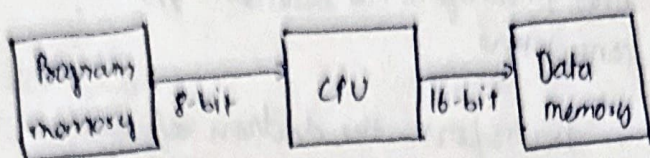
data output line → carry the info out of the memory

Registers

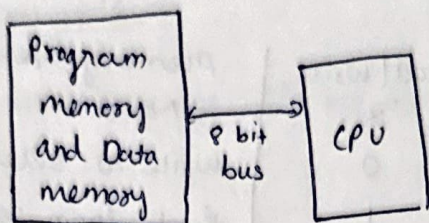
- 1) MAR → Memory address Register → holds the memory location of data that needs to be accessed
- 2) MDR → memory data Register → holds the data that is being transferred to or from memory
- 3) AC → Accumulator → where intermediate arithmetic and logic results are store
- 4) PC → Program Counter → contains the address of the next instruction to be executed
- 5) IR → Instruction Register - contains the current instruction being processed.

~~Process~~

Harvard Architecture [faster as it has 2 memories] [RISC processor]



Von Neuman Architecture [CISC processor]



Endians

- ↳ The way data is stored
- ↳ Big, Little Endian
- ↳ Reading from MSB or LSB

ARM (An advanced RISC machine processor) - Used in mobile devices (the most in Android phones)

- ↳ low power embedded processor
- ↳ low power consumption
- ↳ high density code
- ↳ load store architecture, arithmetic and logical instruction
 - ↓
 - copies data from memory to register file
 - writes the data from the register to the memory
 - ↓
 - access only the register files

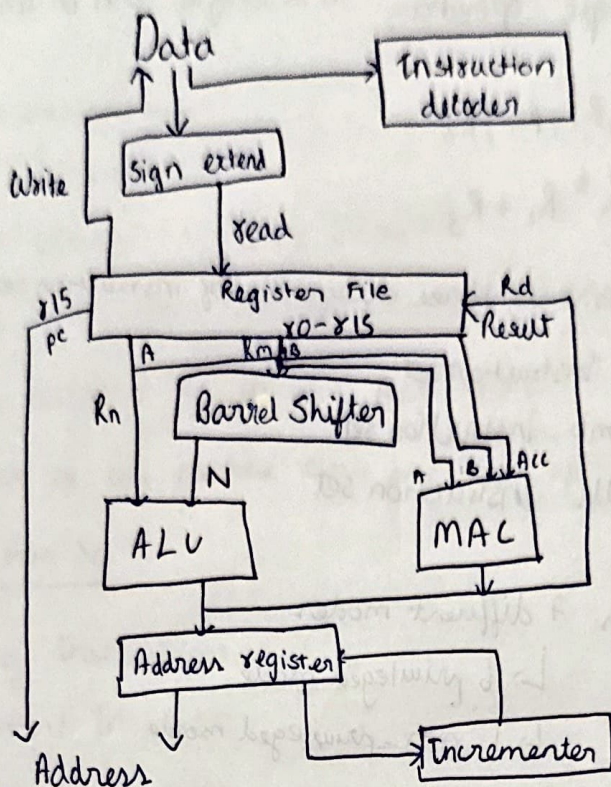
↳ core → 32-bit processor (new version 64)

↳ ARM follows Von Neuman Architecture [Hybrid] → ARM7

↳ ARM7 follows 3-stage pipeline (technique of executing instructions)

↳ ARM7 has 32 registers

Functional Blocks of ARM



Data flow Diagram of ARM

* PC - program counter

contains the address of the next instruction to be executed

* After ~~data~~ the instruction is fetched, the instruction is decoded

* The size of each instruction is 32-bits, a few bits are ~~decoded~~ ARM code (decoded by the decoder)

* Sign extend → It will add 0's if it is a +ve number or 1's if it is a -ve number in the msb, to increase the size to 32 bit if it is less.

* Barrel Shifter → Barrel Shifter is used to pre-shift operand before it is given to ALU. It is combinational logic shifter which can be used for scaled addressing (increase the data size)

* ALU - Control signal unit ~~needs~~ or signal is like a trigger

* Incrementor helps in the array of instructions

*

* MAC (multiply accumulate unit) which has the ability to perform multiple operations in a single unit or instruction.

Ex:- $MLA\ R_0, R_1, R_2, R_3$

$$R_0 = R_1 * R_2 + R_3$$

ARM implements three categories of instruction set:

- * 32-bit ARM Instruction set
- * 16-bit Thumb Instruction set
- * 8-bit Jazelle Instruction set

ARM processor has 7 different modes

- ↳ 6 privileged mode
- ↳ 1 non-privileged mode

1) User mode → unprivileged mode

2) System mode

3) Supervisor

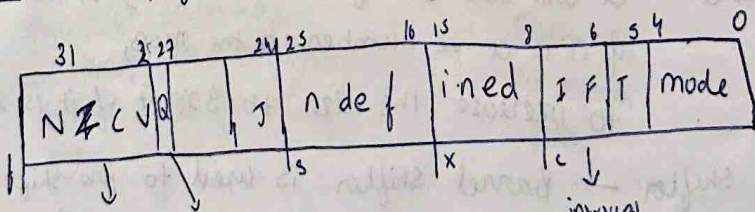
4) Abort

5) Undefined

6) Fiq

7) Hlt.

CPSR (Current program status register)



Conditional sticky
code overflow
flags flag

interrupt
disable
bit

ARM Instruction Set

* Features

- ↳ Load-store architecture
- ↳ 3 address instruction
- ↳ Conditional execution of every instruction
- ↳ Possible to load/store multiple registers at once [In single register to load two data]
- ↳ Possible to combine shift and ALU operations in a single instruction [MIPS have to use 2 instructions]
- ↳ ARM instruction set can combine move and shift operators.

Types of Instruction Set

- 1) Data Processing Instructions
- 2) Data Movement Instructions
- 3) Flow control Instructions

* Data ~~process~~ Processing Instructions

- The Instructions like Arithmetic, Logical, comparison, move and multiply falls under data processing Instructions.
- Most data processing Instructions can process one of their operand using the Barrel shifter.

* Arithmetic operations

* ADD R_0, R_1, R_2

$$R_0 := R_1 + R_2$$

* ADC R_0, R_1, R_2

$$R_0 := R_1 + R_2 + C \rightarrow \text{Add 32 bit values with carry}$$

* SUB R_0, R_1, R_2

$$R_0 := R_1 - R_2$$

(Subtracting from $R_2 - R_1$ in ARM there is an option for $R1 - R2$)

* RSB R_0, R_1, R_2

$$R_0 := R_2 - R_1$$

* SBC R_0, R_1, R_2

$$R_0 := R_1 - R_2 + C - 1 \quad (\text{SBC subtract with c two 32 bit value})$$

→ ARM code sees the operation taking place

* RSC R_0, R_1, R_2 (Reverse subtract with carry)

$$R_0 := R_2 - R_1 + C - 1$$

Logical Instruction set

* ORR R_0, R_1, R_2 (logical OR)

* EOR R_0, R_1, R_2 (logical XOR)

* BIC R_0, R_1, R_2

$$R_0 := R_1 \& \sim R_2$$

Move Instructions

* MOV R_0, R_2

$$R_0 := R_2 \quad (\text{move the content from } R_2 \text{ to } R_0)$$

* MVN R_0, R_2

(move the complement of 32 bit value into the register)

$$R_0 := \sim R_2$$

Compare Instructions

* CMP R_1, R_2 ($R_1 - R_2$)

* CMN R_1, R_2 ($R_1 - \sim R_2$)

* TST R_1, R_2

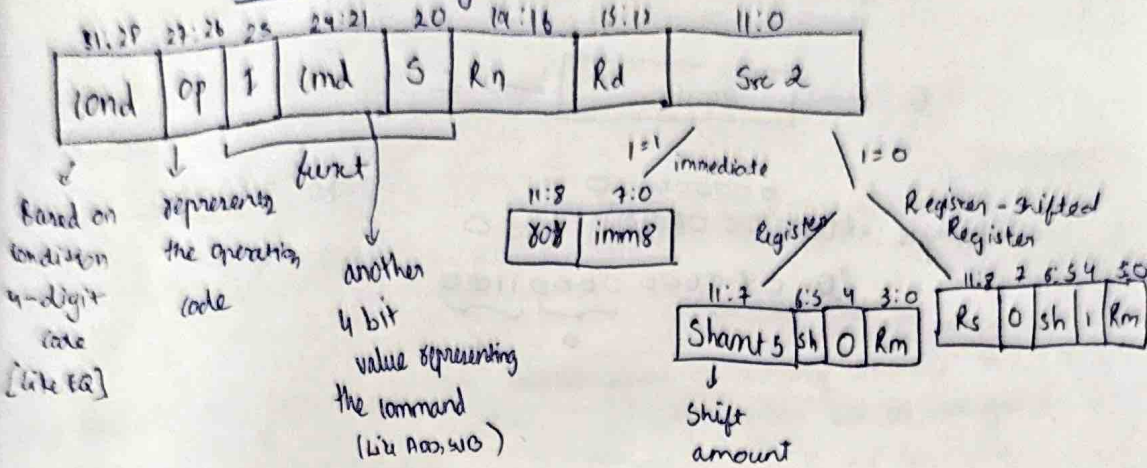
* TEQ R_1, R_2

Note:- Immediate operand - Immediate addressing mode puts the same 32 bit as one of the operands. It replaces the 2nd source operand with an immediate value which is a constant preceded by hash (#) ~~preceded~~

ADD $R_3, R_3, \#35 \rightarrow R_3 := R_3 + 35$

↓
decimal value
raw data

Data processing



* ADD R₀, R₁, R₂ [if no condition take as always (1110)]
 and R_n, R_m

1110 00 0 0000 0 0001 0000 0000 0000 0010

~~5-bit field~~

S-field:- If u use a S suffix on a data processing instructions then it updates the flags in the CPSR (like ADDS R₀, R₁, R₂)
 (Status Bit)

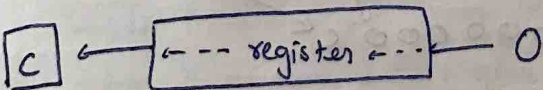
ADC

1. first addition (ADDS R₀, R₁, R₂)

- Add R₁ and R₂ storing the result in R₀
- Set the carry flag (CPSR.c) if an overflow occurs.

2. Second addition

Logical shift left



MOV R₀, R₂, LSL #2 @ R₀ := R₂ << 2
 @ R₂ unchanged

00000000110000
 00000001100000
 C 0

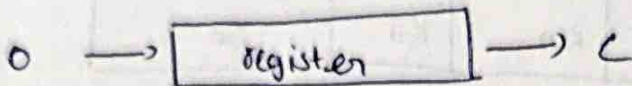
Example: 0...0 0011 0000

Before → R₂ = 0x00000030

After → R₀ = 0x000000C0 (hexadecimal) (after shifting 3 bit become 12 which is converted to hexadecimal)

R₂ = 0x00000030

Logical shift right

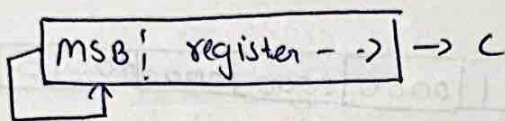


Example :- $00000000 \rightarrow 00000000$

$R0 := R2 \gg 2$

$R0 = 0x00000000$

Arithmetic shift right



$MOV R0 R2 ASR \# 2 @ R0 := R2 \gg 2$

$R2$ unchanged.

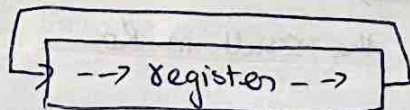
$R2 = 0xA0000030$

$1010\ 0000\ 0011\ 0000\ 0000\ 0000$

$\downarrow$
 111010000000001100

$R0 = 0xE800000C$

Rotate right



$MOV R0 R2 ROR \# 2 @ R0 := R2 \text{ rotate}$

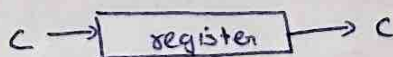
$R2 = 0x00000031$

$00000000\ 0011\ 0001$

$0100\ 0000\ 0000\ 1100$

$R0 = 4000000C$

Rotate right extend



$MOV R0 R2 RRX @ R0 := R2 \text{ rotate}$

$R2 = 0x00000031 \quad C=1$

$00000000\ 0011\ 0001$

$1000\ 0000\ 0001\ 1000$

$R0 = 80000008 \quad C=1$

1) SUBEQS $R_0, R_1, \#2$ [SUB $\rightarrow$ 0010]
 $R_n \quad R_n$ [EQ $\rightarrow$ 0000]

0000	00	1	0010	1	0001	0000	0000	00000010
------	----	---	------	---	------	------	------	----------

[if immediate value is more than 256 then we use (xor) rotate right function]

2) ADD R_0, R_1, R_2 $\rightarrow$ This instruction is wrong because there is no status bit as we get a carry and zero value.

$R_1 = 0 \times \text{FFFFFFFF}$

$R_2 = 0 \times 00000001$

$R_1 =$

1111	1111	1111	1111	1111	1111	1111	1111	1111	1111
0000	0000	0000	0000	0000	0000	0000	0000	0000	0001

$R_2 =$

0000	0000	0000	0000	0000	0000	0000	0000	0000	0000
------	------	------	------	------	------	------	------	------	------

$C=1 \quad Z=1$ [Carry flag is associated with unsigned addition whereas, overflow flag is associated with signed addition]

3) ADD $R_0, R_1, R_2, LSL \#2$
 $R_n \quad R_n \quad R_n$

1110	00	0	0100	0	0001	0000	00010	00	0	0010
------	----	---	------	---	------	------	-------	----	---	------

* Multiplicative operations

~~1) MUL~~

~~2) MLA~~ * MLA - multiplication with accumulator

1) MUL - Normal multiplication

* MUL Instructions multiply two 32 bit numbers held in register and stores the result in a destination register.

* multiplying 2 32-bit numbers give rise to a 64 bit number.

* The MUL Instruction only stores the lower 32 bit of the product

* The ARM also has four Instructions for multiplying 32 bit numbers together and retaining all 64 bits of the product they are :-

UMULL (Unsigned multiplication Long), UMLAL (Unsigned multiplication with accumulator long), SMULL (Signed multiplication long), SMLAL (Signed multiplication with accumulator long)

MUL $R_d, R_m, R_s \rightarrow R_d = (R_m * R_s)$

MLA $R_d, R_m, R_s, R_n \rightarrow R_d = [(R_m * R_s) + R_n]$

~~Load and Store~~

i) If $(Z == 1) \quad R_1 = R_2 + (R_3 * 4)$

~~MUL R1, R3, #4~~

→ multiplication according is similar to Logical left shift

ADD EQS $R_1, R_2, R_3, LSL \#2$

Data - movement Instructions

* Store instruction - Registers $\rightarrow$ main memory

* Load instruction - main-memory $\rightarrow$ Register.

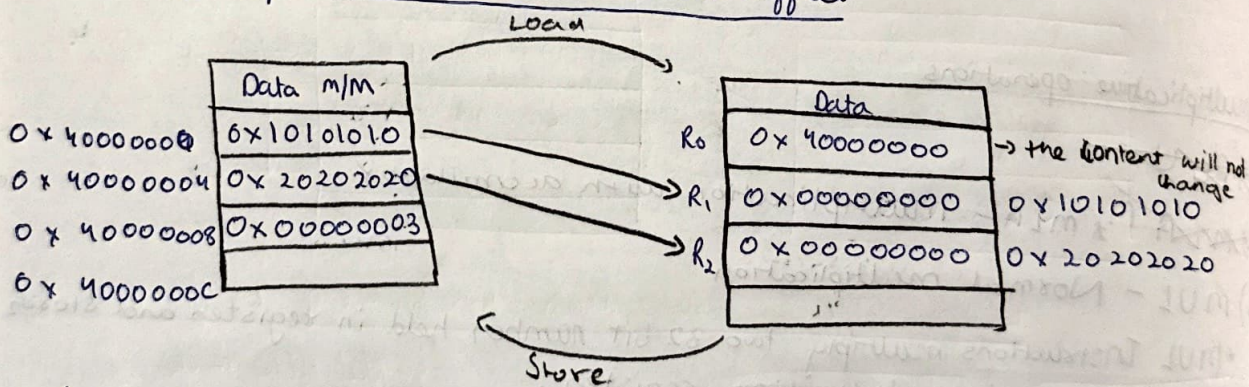
* Load and store instruction are used to move a word, unsigned and signed half word and a byte to and from registers.

* It can also be used to load PC (program counter) (If the target address is beyond branch instruction range)

* LDR (Load register), STR (Store register)

** Single Register Load

Base + Offset preindexed with immediate offset



LDR $R_0 = 0x40000000$

LDR $R_1, [R_0]$

LDR $R_1, [R_0, \#4]$

LDR $R_2, [R_0, \#4] \rightarrow$ It will still point to the same memory address as the change will only to confined to that particular instruction

↓ pre-indexed ↓ Base ↓ offset
(immediate offset)

pre-indexed with immediate

Base + offset pre-indexed with immediate offset and ^{write} ~~data~~ ^{update} ~~back~~

LDR R₀ = 0x40000000

LDR R₁, [R₀]

LDR R₂, [R₀ # 4]!

data m/m
0x10101010
0x20202020
0x00000003

data
0x40000000
0x10101010
0x20202020

→ 0x40000004

Base + offset post-indexed with immediate offset

LDR R₀ = 0x40000000

LDR R₁, [R₀]

LDR R₂, [R₀] # 4

data m/m
0x10101010
0x20202020
0x00000003

data
0x40000000
0x10101010
0x00000000

→ 0x40000004

→ 0x10101010

* LDR R₀ = 0x40000000

LDR R₁, [R₀]

LDR R₂, [R₀] # 4

STP R₂ [R₀ # 4]! updates R₀

data m/m
0x10101010
0x20202020
0x00000003

data
0x40000000
0x00000000
0x00000000

← R₀
← R₁
← R₂
← 40000008

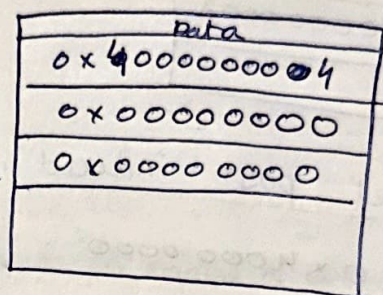
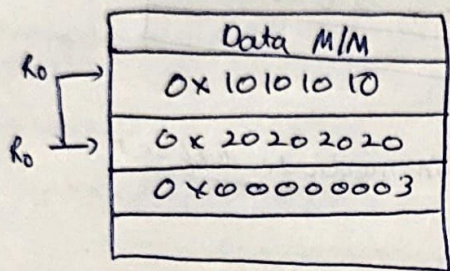
** Single Register Load [Decrement]

LDR R₀ = 0x40000004

LDR R₁, [R₀]

LDR R₂, [R₀ #-4]!

→ decrement the value in R₀ and updates R₀



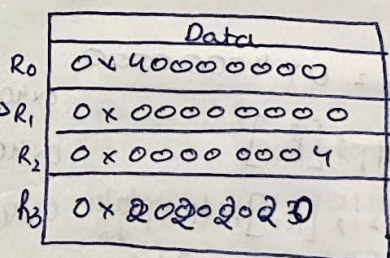
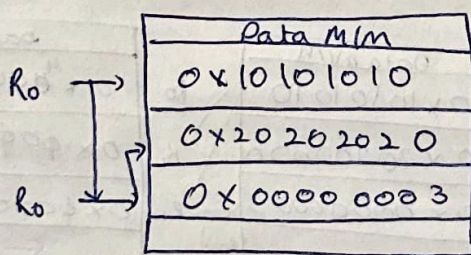
Base + offset pre-indexed with register offset and write back

LDR R₀ = 0x40000000

LDR R₁, [R₀]

LDR R₃, [R₀, R₂]!

→ adding the value of R₀ with R₂



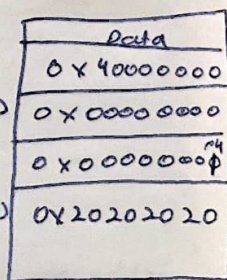
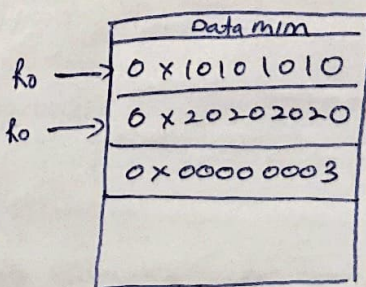
Base + offset pre-indexed with scaled register offset

Scale up → shift left op

LDR R₀ = 0x40000000

LDR R₁, [R₀]

LDR R₃, [R₀, R₂, LSL #2]!



0001 ⇒ 1
0100 ⇒ 4

Control Flow Instructions

↳ These instructions change the order of instruction execution or jump from one memory location to others

Types of Control Flow Instructions

- 1) Unconditional Branch
- 2) Conditional Branch
- 3) Branch and Link

1. Unconditional Branch

Unconditionally loads the pc with specified address.

2. Conditional Branch

Conditionally loads the pc with specified address.

* If $(R2 \neq 10)$ $R5 = R5 + 10 - R3$

CMP $R2 \# 10$ (doesn't need S bit as CMP will automatically update the CPSR register)

BEQ SKIP (Branch if equal)

ADD $R5, R5, R2$

SUB $R5, R5, R3$

→ If condition is true the address of SKIP gets saved in the PC and skips the other operation

else it will execute the other operation

* In ARM there are 4 important branch instructions available they are →

B → Branch

BL → Branch with Link

BX → Branch exchange

BLX → Branch exchange with Link

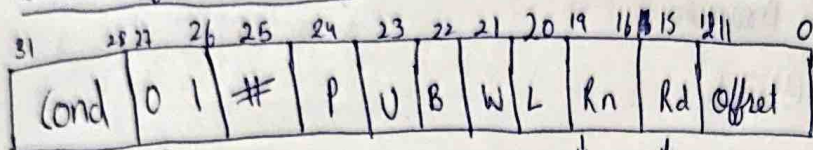
* The BX and BLX instructions change the state of the processor from ARM to Thumb or Thumb to ARM.

* When the Subroutine uses Thumb instruction set and the caller uses the ARM instruction set, the subroutine's ~~code~~ calls and

The processor's working state can be toggled through the BLX instruction.

* At the same time, the return of the subroutine can be done by copying the register R_{14} value to R_{15} (PC)

→ Data transfer Instructions



↓
Op
code

↓
offset
value

Base reg.
-sten

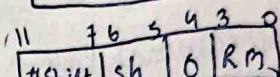
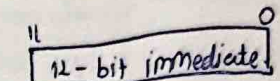
destination in load, source in store

= 0

= 1

[immediate
value] → $R_2[R_0 \# 4]$

[It is 1 when it
is a register] → $R_2[R_0 R_3 \text{ 2SL} \# 4]$



↓
Shift
amount

↓
type
shift

↓
offset
register

Flag bits

• L → Single bit, represent if it is Load or store

L = 0 → Store instruction

L = 1 → Load instruction

00	-L32
01	-L32R
10	-ASR
11	-RR

• W → W = 0 → no write back

W = 1 → write back

[!]

• B → unsigned byte or word

B = 0 → word

B = 1 → byte

• U → increment or decrement (up or down)

U = 0 → subtract offset from base

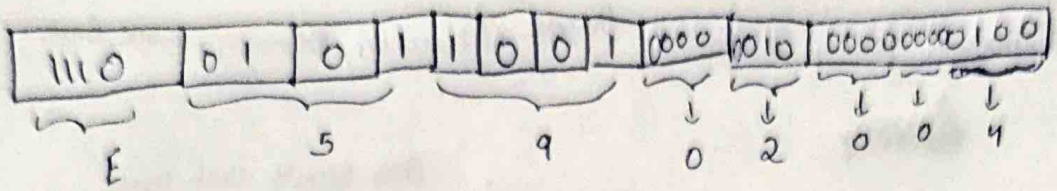
U = 1 → add offset to the base

• P → postfix or prefix [post-index or pre-index]

P = 0 → Post

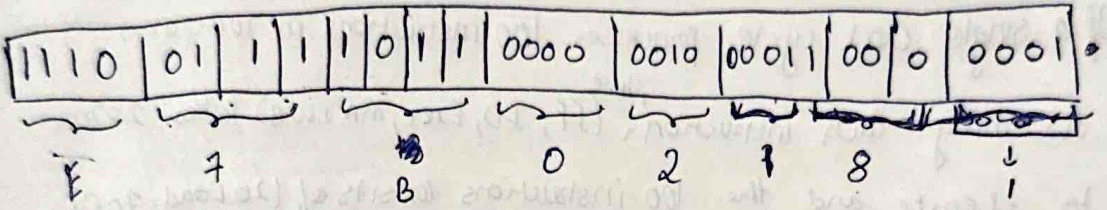
P = 1 → Pre

Q) LDR R₂[R₀ #4]



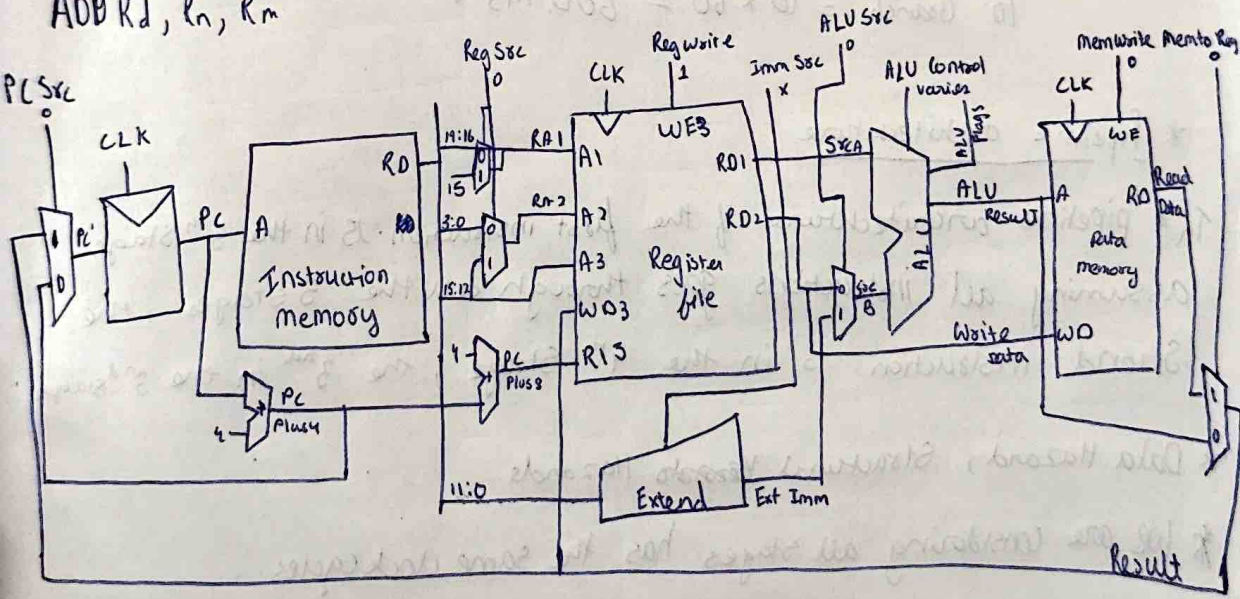
⇒ 0x F5902004 → hexadecimal format

Q) LDR R₂[R₀, R₁, LSL #3]!



⇒ 0x E7B0281

*Write the single cycle data path diagram for SUB R_d R_n R_m
ADD R_d, R_n, R_m



* Instruction fetch, Instruction Decode, Execution, Memory Access, Write Back

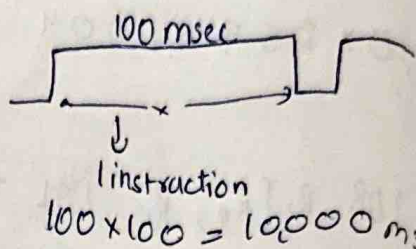
~~Branch~~

Branch - IF, ID, Exec (fast)

ADD - IF, ID, Exec, WB

LOAD - IF, ID, Exec, MA, WB

Single clock cycle



If a single clock cycle processes 100 instructions in 10000 ms

Assuming each instruction ^{phase} (IF, ID, Exec, MA, WB) takes 20 ms to execute and the 100 instructions consists of (20 Load, 70 OP, 10 Branch), a multi clock cycle takes

$$20 \text{ Load} = 20 \times 100 = 2000 \text{ ms}$$

$$70 \text{ OP} = 70 \times 80 = 5600 \text{ ms}$$

$$10 \text{ Branch} = 10 \times 60 = 600 \text{ ms}$$

$$\left. \begin{array}{l} 2000 \text{ ms} \\ 5600 \text{ ms} \\ 600 \text{ ms} \end{array} \right\} 8200 \text{ ms}$$

* Pipeline architecture

In pipeline architecture if the first instruction is in the 5th stage, assuming all instructions goes through all the 5 stages, the Second instruction is in the 4th Stage, the 3rd in the 3rd stage

* Data Hazard, Structural Hazards

* We are considering all stages has the same clock cycles.